BIG BEN GLOBAL LIMITED

AI-POWERED E-COMMERCE WEBSITE • TWO-PAGE BUILD PROMPT

MASTER INSTRUCTION — COPY THIS PROMPT INTO YOUR AI WEBSITE BUILDER

Act as a senior product strategist, UX/UI designer, full-stack engineer, database architect, cybersecurity specialist and AI integration engineer. Plan and build a production-minded, responsive business website and e-commerce system for **Big Ben Global Limited**, a company that sells **beetroot tea**. Do not create only a static mock-up. Build a usable end-to-end application with working navigation, authentication, product catalogue, cart, checkout flow, customer account area, admin dashboard, contact handling and an AI chatbot. Make sensible decisions, explain important trade-offs and solve problems rather than blindly following instructions.

1. BUSINESS GOALS & CUSTOMER VALUE

- **Primary goal:** turn visitors into informed customers and make it easy to discover, understand and purchase Big Ben Global Limited beetroot tea.
- Communicate the product clearly: ingredients, preparation instructions, pack sizes, price, availability, storage, delivery information and answers to common questions. Use only company-confirmed facts.
- Do not invent medical claims, disease-treatment promises, certifications, customer reviews, clinical evidence or product benefits. Use cautious, compliant wellness language; label unverified details as placeholders for approval.
- Design for trust: transparent pricing, clear policies, secure checkout, accessible contact details, genuine product photography and honest FAQs.
- Use a modern natural-wellness visual direction: beetroot/burgundy accents, leafy green, warm cream and charcoal. Use strong contrast, readable typography, generous spacing and mobile-first layouts.

2. REQUIRED PUBLIC PAGES

- **Home:** compelling hero headline and product image, concise value proposition, Shop Now and Learn More calls to action, product highlights, how to prepare the tea, trust signals, FAQs, testimonials only when supplied and newsletter sign-up.
- **About:** company story, mission, values, product sourcing/quality approach and team or company details. Use editable placeholders where facts are not supplied.
- **Shop / Product detail:** product images, variants/pack sizes, price, availability, quantity selector, ingredients, preparation, storage, shipping/returns information and Add to Cart. Include search/filtering if multiple products are added.
- **Contact:** validated contact form, business email/phone/address placeholders, business hours, map only when a verified address is provided, and clear success/error states.
- **Login / Sign up:** secure registration, login, logout, password reset, email verification and account profile. Include clear validation and helpful error messages.
- **Cart / Checkout:** editable cart, order summary, shipping details, delivery fee calculation, payment step, order confirmation and order reference. Never collect or store raw card details.
- **Customer dashboard:** profile, saved addresses, order history, order status, receipts where supported and account security settings.

3. DASHBOARD & ADMIN WORKSPACE

- Create role-based access: customers see only their own account and orders; authorised administrators access the admin dashboard.
- Admin overview: sales totals, order counts, pending orders, recent activity and low-stock alerts. Use real database values, not decorative sample metrics in production.
- Admin tools: create/edit/archive products, manage images, prices, variants and inventory; view/update orders; manage customers; review contact enquiries; manage FAQs/chatbot knowledge; configure shipping, tax and payment settings.
- Add search, filters, pagination, confirmation dialogs, loading/empty/error states and audit logs for important admin actions.

BUILD SPECIFICATION & ACCEPTANCE CRITERIA

BIG BEN GLOBAL LIMITED • FUNCTIONALITY, AI, ARCHITECTURE & QUALITY

4. AI CHATBOT — USEFUL, GROUNDED & SAFE

- Add a visible chat launcher across the website. Answer questions about beetroot tea, approved product details, preparation, prices, stock, delivery, returns, orders and company information.
- Ground answers in an editable, administrator-managed knowledge base (approved product data, FAQs and policies). If information is missing or uncertain, say so and offer the contact form or human support; never fabricate facts.
- Recommend relevant products based on stated preferences, link to product pages and guide users through the cart. Require login and verification before revealing private order information.
- For order lookups, securely verify the customer and retrieve only their own order. Do not expose personal data, payment details or admin functions in chat.
- Add privacy notice, conversation controls, rate limiting, abuse protection, fallback when the AI service is unavailable and an admin view of unresolved questions.
- Use an AI provider through a server-side API. Store secrets in environment variables, never in browser code. Log only necessary data and provide appropriate chat-history controls.

5. TECHNICAL IMPLEMENTATION

- **Frontend:** responsive, accessible interface using a modern framework such as Next.js/React and TypeScript. Use reusable components, design tokens, form validation and clear feedback states.
- **Backend:** secure server-side APIs for authentication, products, inventory, cart, orders, payments, contact enquiries, chatbot requests and admin operations. Validate inputs on the server.
- **Database:** PostgreSQL or equivalent relational database with schema/migrations for users, roles, products, variants, inventory, carts, orders, order items, addresses, payments, enquiries, chatbot knowledge and audit events.
- **Authentication & security:** trusted auth solution, secure sessions/cookies, role-based access, CSRF/XSS/SQL-injection protections, rate limits, server-side authorization and least-privilege database access.
- **Payments:** integrate a reputable gateway suitable for the target market (e.g. Paystack or Flutterwave, subject to business account approval). Use secure hosted flows, verify status server-side via signed webhooks and make order creation idempotent.
- **Operations:** provide environment-variable examples, setup instructions, seed/demo data clearly marked as demo, migrations, error monitoring, backups, deployment instructions and README. Never expose secrets or production customer data.

6. PROBLEM-SOLVING & DELIVERY PROCESS

- First identify missing business inputs: approved product description, pack sizes, prices, images, ingredients, claims, policies, contact details, shipping zones and payment account. Use clearly labelled placeholders rather than inventing them.
- Before coding, provide a concise sitemap, user journeys, component plan, database model, API route list, security approach and implementation milestones. Flag risks and recommend practical solutions.
- Build in increments: public pages → catalogue → authentication → cart/checkout → customer dashboard → admin dashboard → chatbot → testing and deployment.
- Test on mobile, tablet and desktop. Add tests for authentication, permissions, cart totals, inventory, order creation, payment webhooks and chatbot privacy. Fix errors found; do not claim a feature works unless tested.
- Include keyboard navigation, labels, focus states, alt text, SEO metadata, fast image loading, performance optimisation and clear 404/500 pages.

7. DEFINITION OF DONE

Deliver a running application—not just screenshots—with working links and forms, persistent database records, protected dashboards, real product/cart/order flows, secure payment integration in test mode, and a functioning AI chatbot connected to approved business information. Provide complete source code, environment-variable template, database setup/migrations, test instructions and deployment guide. Clearly list anything requiring company credentials, approvals or final content before launch.

Final instruction: Prioritise reliability, customer trust, security and conversion over visual effects. Use realistic demo data only where necessary and label it clearly. Ask focused questions only when a missing decision blocks implementation; otherwise proceed with safe, documented defaults.